

【开源】轮腿 TinyMPC 与碰撞检测

从力矩约束到 STM32H7 实现

变腿长模型、ADMM 求解与广义动量观测器

一、 引言

这篇帖子拖到现在才整理出来，主要是前段时间忙于推免相关工作。去年就有过在嵌入式平台上部署 MPC 的想法，但一直受限于单片机的计算资源。今年接触 TinyMPC 后，终于把这件事做了出来，也取得了一些实际效果。这里以轮腿机器人为例，记录模型建立、参数离线生成、STM32H7 部署和 TinyMPC 的使用方法。此外还介绍基于广义动量观测器的碰撞检测：根据各轴残差判断偏航和腿部堵转，再把检测结果接入脱困与参考调整。

二、 为什么在 LQR 之外加入 MPC

最初想换控制算法，主要是高腿时偶发的不稳定。腿抬高以后，行进或受扰造成的姿态偏差会带来更大的轮端力矩需求。LQR 的反馈算得很快，但算出的力矩有时已经超过电机能力，只能在发送前截到安全限幅内。

$$u_{\text{LQR}} = -K(x - x_{\text{ref}}), \quad u_{\text{act}} = \text{clip}(u_{\text{LQR}}, u_{\text{min}}, u_{\text{max}}).$$

传统无约束 LQR 的增益是在输入不受限的条件下设计的。连续几拍被截断后，机器人实际得到的输入已经偏离这套反馈，原来的闭环响应也会改变。高腿大扰动时，这种偏差可能持续扩大，最后失衡。就如图中的仿真所示起步时轮力矩远超电机实际峰值。导致电机无法产生最优输出，误差积累最终失衡。

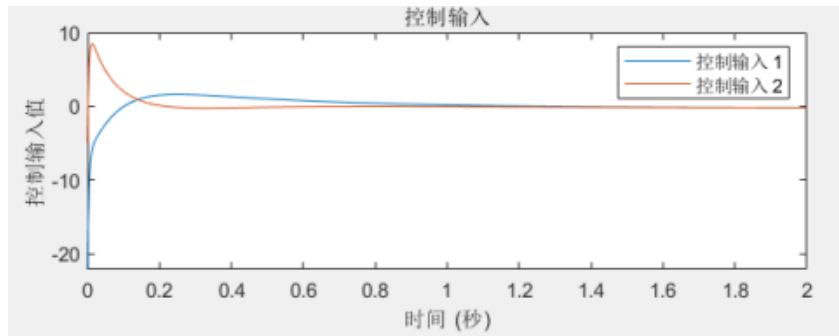


图 1 高腿起步时的控制力矩仿真曲线

调小操作期望、提高某些状态的权重，都能改善部分工况。不过外部冲击不是操作期望能完全覆盖的，提高一个通道的权重也会改变其他通道的权衡。我更希望控制器在算力矩时，就把执行器边界一起考虑进去。

要在计算力矩时考虑这些边界，可以把问题写成 MPC。它从当前状态出发，预测接下来一段时间的状态和输入，在跟踪目标的同时要求每段输入落在给定范围内。它可以提前权衡轮与关节的出力，以及速度、姿态等状态的偏差，而不是得到一个过大的控制量后再统一截断。

$$x_{k+1} = A_d x_k + B_d u_k + f_d,$$
$$x_{\text{min}} \leq x_k \leq x_{\text{max}}, \quad u_{\text{min}} \leq u_k \leq u_{\text{max}}.$$

每次执行当前求解结果的首个输入，下一拍再根据新状态滚动求解。和在线只执行状态反馈的传统 LQR 相比，这里显式规划了带约束的后续轨迹。LQR 的设计本身也来自模型和最优控制，所以更准确的区别在于在线预测与约束处理。

对这台轮腿来说，引入 MPC 的主要目的就是缓解高腿受扰后的力矩饱和。模型误差、求解未收敛和结果过期仍要处理，带约束的优化本身不能保证所有实车工况都稳定。

约束加进来了，在线计算量也随之增加。LQR 主要是一次矩阵乘法，MPC 则要在每拍求解一个优化问题。预测节点增加后，要处理的状态和输入也随之增加；腿长变化还会改变模型和求解器矩阵。若把建模、矩阵分解和迭代全放在线做，很难留出稳定的控制周期。

前期我也考虑过用 OSQP 来解这个问题。按当时的工程安排，比较现实的方式是放到上位机运行，但这样需要和已有视觉程序协调，还会多出一段通信链路，最后还是希望把 MPC 留在 MCU 内完成。

三、 TinyMPC 原理（怎样减少在线计算）

下面简单介绍 TinyMPC 怎样利用优化问题的结构来减少在线计算，这也有助于理解后面的程序实现。先从常见的 MPC 轨迹跟踪代价函数出发：

$$J = \sum_{k=0}^{N-1} \frac{1}{2} (x_k - x_k^{\text{ref}})^{\top} Q (x_k - x_k^{\text{ref}}) + \frac{1}{2} (u_k - u_k^{\text{ref}})^{\top} R (u_k - u_k^{\text{ref}}) + \frac{1}{2} (x_N - x_N^{\text{ref}})^{\top} Q_f (x_N - x_N^{\text{ref}}).$$

把单个跟踪误差项展开后，其中的常数项不会影响最优解，可以在求解时省去：

$$\frac{1}{2} (x - r)^{\top} Q (x - r) = \frac{1}{2} x^{\top} Q x + q^{\top} x + \underbrace{\frac{1}{2} r^{\top} Q r}_{\text{常数项}}, \quad q = -Qr.$$

省去不影响最优解的常数项后，MPC 可以整理成下面的二次规划形式：

$$\begin{aligned} \min_{x_{1:N}, u_{0:N-1}} \quad & \frac{1}{2} x_N^{\top} Q_f x_N + q_f^{\top} x_N \\ & + \sum_{k=0}^{N-1} \left[\frac{1}{2} x_k^{\top} Q x_k + q_k^{\top} x_k + \frac{1}{2} u_k^{\top} R u_k + r_k^{\top} u_k \right] \\ \text{s.t.} \quad & x_{k+1} = A x_k + B u_k, \quad x_k \in \mathcal{X}, \quad u_k \in \mathcal{U}. \end{aligned}$$

为了方便处理状态和输入的范围约束，引入辅助变量 z 、 w 作为预测状态和输入的副本，再使用指示函数表示集合限制：

$$\begin{aligned} x_k &= z_k, \quad u_k = w_k, \quad z_k \in \mathcal{X}, \quad w_k \in \mathcal{U}, \\ \min \quad & J(x, u) + I_{\mathcal{X}}(z) + I_{\mathcal{U}}(w), \\ \text{s.t.} \quad & x_{k+1} = A x_k + B u_k, \quad x_k = z_k, \quad u_k = w_k, \\ & I_{\mathcal{X}}(z) = \begin{cases} 0, & z \in \mathcal{X}, \\ +\infty, & z \notin \mathcal{X}. \end{cases} \end{aligned}$$

对于这个带约束的优化问题，如果直接构造并求解完整的 KKT 方程组，计算量可能比较大。TinyMPC 利用了 ADMM 的思路，将轨迹优化和约束处理拆开。这样，轨迹优化部分可以利用

Riccati 递推求解，状态和输入的范围约束则通过投影处理。

$$\mathcal{L}_\rho = f(x) + I_C(z) + \lambda^\top(x - z) + \frac{\rho}{2}\|x - z\|_2^2.$$

带入前面的 MPC 代价函数, 就得到这个方程:

$$\begin{aligned}\mathcal{L}_\rho &= J(x, u) + I_{\mathcal{X}}(z) + I_{\mathcal{U}}(w) \\ &+ \sum_k [\lambda_k^\top(x_k - z_k) + \frac{\rho}{2}\|x_k - z_k\|^2] \\ &+ \sum_k [\mu_k^\top(u_k - w_k) + \frac{\rho}{2}\|u_k - w_k\|^2].\end{aligned}$$

接着把平方项展开、合并同类项，状态和输入的二次项及线性项可以整理成：

$$\begin{aligned}\tilde{Q} &= Q + \rho I, & \tilde{R} &= R + \rho I, \\ \tilde{q}_k &= q_k + \lambda_k - \rho z_k, & \tilde{r}_k &= r_k + \mu_k - \rho w_k.\end{aligned}$$

这样一来，每轮迭代中负责轨迹计算的部分，就成了一个仅含动力学等式约束的二次子问题。原来的范围约束留给后面的投影步骤处理。这个二次子问题具有 LQR 型结构，正好可以用 Riccati 递推来求解。

$$\begin{aligned}\min_{x, u} \quad & \frac{1}{2}x_N^\top \tilde{Q}_f x_N + \tilde{q}_f^\top x_N \\ & + \sum_{k=0}^{N-1} \left[\frac{1}{2}x_k^\top \tilde{Q} x_k + \tilde{q}_k^\top x_k + \frac{1}{2}u_k^\top \tilde{R} u_k + \tilde{r}_k^\top u_k \right] \\ \text{s.t.} \quad & x_{k+1} = Ax_k + Bu_k.\end{aligned}$$

对于给定的模型、权重和惩罚参数，Riccati 递推涉及的一部分矩阵可以预先算好。在线阶段再利用这些结果更新预测轨迹，相关递推关系如下：

$$\begin{aligned}\tilde{Q} &= Q + \rho I, & \tilde{R} &= R + \rho I, \\ H_k &= \tilde{R} + B^\top P_{k+1} B, \\ K_k &= H_k^{-1} B^\top P_{k+1} A, \\ P_k &= \tilde{Q} + A^\top P_{k+1} (A - BK_k), \\ u_k &= -K_k x_k - d_k, & x_{k+1} &= Ax_k + Bu_k.\end{aligned}$$

轨迹算出来之后，还要把输入和状态限制在允许范围内。先将对偶变量改写为缩放形式：

$$y = \frac{\lambda}{\rho}, \quad g = \frac{\mu}{\rho}.$$

状态副本的更新可以进一步写成投影问题：

$$I_{\mathcal{X}}(z) + \lambda^{j+1}^\top(x^{j+1} - z) + \frac{\rho}{2}\|x^{j+1} - z\|_2^2.$$

对盒约束而言，只需逐元素投影到允许区间：

$$z^{j+1} = \arg \min_{z \in \mathcal{X}} \|x^{j+1} + y^j - z\|_2^2.$$

随后更新对偶变量，重复上述步骤，直到残差满足收敛条件或达到迭代上限：

$$\begin{aligned} y_k^{j+1} &= y_k^j + x_k^{j+1} - z_k^{j+1}, \\ g_k^{j+1} &= g_k^j + u_k^{j+1} - w_k^{j+1}. \end{aligned}$$

到这里就能看出它适合 MCU 的原因了。对于固定的模型与参数，Riccati 所需的增益、矩阵分解等工作可以尽量提前算好；在线阶段主要做矩阵乘加、约束投影和对偶变量更新，达到收敛条件或迭代上限后输出结果。这样每拍就不必重新求解一个通用的大规模 QP。

四、 轮腿模型简述

这里简述本工程的建模方式，方便代码阅读。我考虑了质心分布的影响对上交并联腿模型做了一些变化 [3]

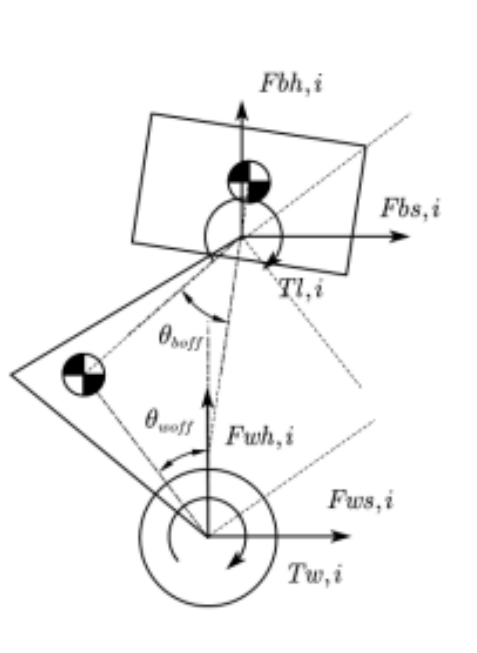


图 2 考虑腿部质心偏移的轮腿受力模型

腿部动力学中考虑了质心偏移，关系式如下：

$$\begin{aligned} I_{l,l} + \ddot{\theta}_{l,l} &= F_{wh,l} * I_{w,l} * \sin(\theta_{woff}) + F_{bh,l} * I_{b,l} * \sin(\theta_{boff}) - F_{ws,l} * I_{w,l} * \cos(\theta_{woff}) \\ &\quad - F_{bs,l} * I_{b,l} * \cos(\theta_{boff}) - T_{w,l} - T_{b,l} \\ I_{l,r} + \ddot{\theta}_{l,r} &= F_{wh,r} * I_{w,r} * \sin(\theta_{woff}) + F_{bh,r} * I_{b,r} * \sin(\theta_{boff}) - F_{ws,r} * I_{w,r} * \cos(\theta_{woff}) \\ &\quad - F_{bs,r} * I_{b,r} * \cos(\theta_{boff}) - T_{w,r} - T_{b,r} \end{aligned}$$

质心位置与转动惯量由 CAD 软件导出，供不同腿长的动力学建模和参数拟合使用。下面是数据格式示例：

dist_mm	com_y_mm	com_x_mm	Ixx_kgmm2
120	-4.838375	82.414434	13867.3636
180	-34.860329	81.032639	14482.6976
240	-59.308431	75.950794	15168.3394
300	-81.648078	67.831895	16012.2939
360	-102.993416	55.731578	17035.4147
406	-118.956213	41.832270	17944.5429

表 1 CAD 导出的腿部质心与惯量数据示例
字段含义依次为腿长、相对髋关节的竖直/水平质心坐标及腿部总惯量。

五、 实现流程

真正开始移植时，我把工作分成两块：MATLAB 负责按腿长生成模型和求解器参数，H7 负责在线更新状态并运行求解器。在接入自己的轮腿代码之前，最好先把 TinyMPC 官方例程在电脑上编译、跑通，方便后面分清是求解器的问题还是移植的问题。

TinyMPC 官方仓库：<https://github.com/TinyMPC/TinyMPC> 官方工程中与求解有关的文件主要在下面这些位置，后面修改时会经常用到：

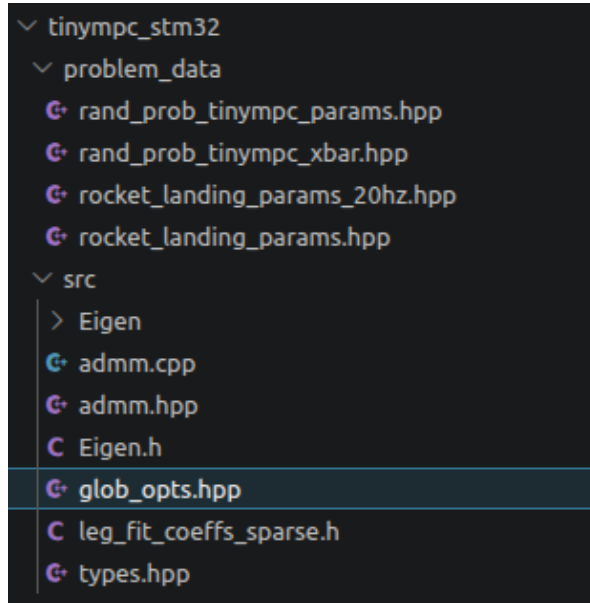


图 3 TinyMPC 工程目录与相关文件

结合前面的推导，求解器需要用到一批可提前计算的矩阵。我这里采用 10 维状态、4 维输入，相关数据及尺寸列在下表。要注意，离线 Riccati 计算使用的状态和输入权重，需要包含 ADMM 惩罚参数 对应的项。

量	尺寸	作用和导出方式
Ad	10×10	离散状态矩阵, 按腿长拟合
Bd	10×4	离散输入矩阵, 按腿长拟合
fd	10×1	离散重力仿射项, 按腿长拟合
u_eq	4×1	当前工作点静态前馈, 按腿长拟合
K	4×10	Riccati 反馈增益, 按腿长拟合
K_{∞}	10×10	不直接拟合 P, 拟合 Cholesky 下三角 Lp
P_{∞}	4×4	不直接拟合逆矩阵, 拟合 Cholesky 下三角 Lq
AmBKt	10×10	H7 按 (Ad-Bd*K)' 重建, 不单独拟合

表 2 按腿长导出的 TinyMPC 参数及尺寸

生成的.cpp 文件里会用到这些数据。由于轮腿的模型随腿长变化, 不能只放一套固定系数; 我在 MATLAB 里按腿长生成并拟合参数, 再让 H7 根据当前腿长更新。下面是部分导出的矩阵:

```
// State transition matrix A (10x10)
tinytype Adyn_data[NSTATES * NSTATES] = {
    // Row 0
    1.0000f, 0.0040f, 0.0f,    0.0f,    -0.0004f, 0.0f,    -0.0004f, 0.0f,    0.0f,    0.0f,
    // Row 1
    0.0f,    1.0000f, 0.0f,    0.0f,    -0.1949f, -0.0004f, -0.1949f, -0.0004f, 0.0f,    0.0f,
    // Row 2
    0.0f,    0.0f,    1.0000f, 0.0040f, 0.0f,    0.0f,    0.0f,    0.0f,    0.0f,    0.0f,
    // Row 3
    0.0f,    0.0f,    0.0f,    1.0000f, -0.0163f, 0.0f,    0.0163f, 0.0f,    0.0f,    0.0f,
    // Row 4
    0.0f,    0.0f,    0.0f,    0.0f,    1.0022f, 0.0040f, 0.0001f, 0.0f,    0.0f,    0.0f,
    // Row 5
    0.0f,    0.0f,    0.0f,    0.0f,    1.1101f, 1.0022f, 0.0580f, 0.0001f, 0.0f,    0.0f,
    // Row 6
    0.0f,    0.0f,    0.0f,    0.0f,    0.0001f, 0.0f,    1.0022f, 0.0040f, 0.0f,    0.0f,
    // Row 7
    0.0f,    0.0f,    0.0f,    0.0f,    0.0580f, 0.0001f, 1.1101f, 1.0022f, 0.0f,    0.0f,
    // Row 8
    0.0f,    0.0f,    0.0f,    0.0f,    0.0f,    0.0f,    0.0f,    0.0f,    1.0001f, 0.0040f,
    // Row 9
    0.0f,    0.0f,    0.0f,    0.0f,    -0.0161f, 0.0f,    -0.0161f, 0.0f,    0.0277f, 1.0001f
};
```

图 4 导出的离散状态矩阵示例

```
// Control matrix B (10x4)
tinytype Bdyn_data[NSTATES * NINPUTS] = {
    // Row 0
    0.0001f, 0.0001f, 0.0f, 0.0f,
    // Row 1
    0.0322f, 0.0322f, -0.0048f, -0.0048f,
    // Row 2
    0.0f, 0.0f, 0.0f, 0.0f,
    // Row 3
    -0.0142f, 0.0142f, -0.0004f, 0.0004f,
    // Row 4
    -0.0003f, 0.0f, 0.0001f, 0.0f,
    // Row 5
    -0.1641f, -0.0137f, 0.0275f, 0.0014f,
    // Row 6
    0.0f, -0.0003f, 0.0f, 0.0001f,
    // Row 7
    -0.0137f, -0.1641f, 0.0014f, 0.0275f,
    // Row 8
    0.0f, 0.0f, 0.0f, 0.0f,
    // Row 9
    0.0007f, 0.0007f, -0.0075f, -0.0075f
};
```

图 5 导出的输入矩阵示例

```
tinytype Pinf_data[NSTATES * NSTATES] = {
    // Row 0
    70057.0f, 59986.0f, 0.0f, 0.0f, 19124.0f, 10916.0f, 19124.0f, 10916.0f, 5880.0f, 3364.0f,
    // Row 1
    59986.0f, 99770.0f, 0.0f, 0.0f, 22532.0f, 18159.0f, 22532.0f, 18159.0f, 7293.0f, 5617.0f,
    // Row 2
    0.0f, 0.0f, 10404.0f, 895.0f, -995.0f, -63.0f, 995.0f, 63.0f, 0.0f, 0.0f,
    // Row 3
    0.0f, 0.0f, 895.0f, 375.0f, -377.0f, -26.0f, 377.0f, 26.0f, 0.0f, 0.0f,
    // Row 4
    19124.0f, 22532.0f, -995.0f, -377.0f, 82946.0f, 4728.0f, 5454.0f, 4035.0f, 3424.0f, 1358.0f,
    // Row 5
    10916.0f, 18159.0f, -63.0f, -26.0f, 4728.0f, 3325.0f, 4035.0f, 3303.0f, 1399.0f, 1029.0f,
    // Row 6
    19124.0f, 22532.0f, 995.0f, 377.0f, 5454.0f, 4035.0f, 82946.0f, 4728.0f, 3424.0f, 1358.0f,
    // Row 7
    10916.0f, 18159.0f, 63.0f, 26.0f, 4035.0f, 3303.0f, 4728.0f, 3325.0f, 1399.0f, 1029.0f,
    // Row 8
    5880.0f, 7293.0f, 0.0f, 0.0f, 3424.0f, 1399.0f, 3424.0f, 1399.0f, 29567.0f, 1622.0f,
    // Row 9
    3364.0f, 5617.0f, 0.0f, 0.0f, 1358.0f, 1029.0f, 1358.0f, 1029.0f, 1622.0f, 428.0f
};
```

图 6 导出的 Riccati 缓存矩阵示例

```
// Pre-computed Quu inverse (4x4)
tinytype Quu_inv_data[NINPUTS * NINPUTS] = {
    // Row 0
    0.3831, -0.0100, 0.0281, 0.0002,
    -0.0100, 0.3831, 0.0002, 0.0281,
    0.0281, 0.0002, 0.8939, -0.0020,
    0.0002, 0.0281, -0.0020, 0.8939
};
```

图 7 导出的输入权重逆矩阵示例

这些参数我用 MATLAB 统一生成。采样周期、预测长度、权重、惩罚参数、动力学和五连杆尺寸等，都集中放在 wheelleg_parameters.m 中配置；通常需要修改的几个位置如下：

位置	需要修改的内容
matlab/wheelleg_parameters.m	Ts、N、Q、R、rho、约束、动力学参数、五连杆尺寸和输出路径
matlab/data/leg_mass_properties.csv	各虚拟腿长对应的 CAD 质心坐标和腿部惯量
matlab/generate_mpc_fit.m	唯一生成入口；正常使用时不在此处填写参数
H7 tinympc_task.cpp	状态来源、参考值、故障保护、任务频率和电机映射

表 3 MATLAB 与 H7 常用修改位置

从 generate_mpc_fit.m 进入后，脚本会依次处理 CAD 数据、动力学、连续模型采样、离散化以及参数拟合和导出。想读懂这部分代码，按下面的调用顺序往下看就行：

文件	职责
generate_mpc_fit.m	唯一主入口，按 4 个阶段顺序调用所有步骤
wheelleg_parameters.m	Q/R/rho、采样周期、约束、整车参数、五连杆和路径
prepare_leg_data.m	读取 CSV，计算腿长、质心距离、偏角和惯量
derive_reduced_dynamics.m	写受力方程并消去内力，得到 M、B_tau、G
derive_ab_model.m/derive_gravity_model.m	分别生成连续 A/B 数值函数和重力 G 数值函数
sample_ab_models.m	遍历左腿点 × 右腿点，得到连续 Ac/Bc/fc
generate_mpc_parameters.m	ZOH、Riccati、二维拟合和 C 头文件导出

表 4 MATLAB 参数生成脚本及职责

MATLAB 导出完成后，就轮到 H7 侧的配置了。我移植时主要检查了几个地方：求解器维度是否一致、Cache 和 DMA 内存安排有没有冲突，以及初始化有没有漏掉参数。

首先要核对 glob_opts.hpp 里的宏定义。NSTATES、NINPUTS、NHORIZON 必须和 MATLAB 一致；我这里分别是 10、4、8。NTOTAL 设置迭代次数上限，NUM_INPUT_CONES 和 NUM_STATE_CONES 对应输入、状态的锥约束数量。

```

2026_H7NewFrame > TDT_Alg > tinympc_stm32 > src > glob_opts.hpp > ...
1  #pragma once
2
3  typedef float tinytype; // should be double if you want to generate code
4
5  #define NSTATES 10
6  #define NINPUTS 4
7
8  #define NUM_INPUT_CONES 1
9  #define NUM_STATE_CONES 1
10
11 #define NHORIZON 8
12 #define NTOTAL 100
13

```

图 8 求解器维度与预测时域配置

然后是 H7 的 Cache。这个细节很容易漏掉：即使主频已经配到 550 MHz，如果 I-Cache 和 D-Cache 没有开启，TinyMPC 中的矩阵运算也可能明显变慢。我在 main() 初始化时先配置 MPU，再开启 CPU Cache，代码如下：

```

int main(void)
{
    /* USER CODE BEGIN 1 */

    /* USER CODE END 1 */

    /* MPU Configuration-----*/
    MPU_Config();
    /* Enable the CPU Cache */

    /* Enable I-Cache-----*/
    SCB_EnableICache();

    /* Enable D-Cache-----*/
    SCB_EnableDCache();

    /* MCU Configuration-----*/

```

图 9 STM32H7 的 MPU 与 Cache 初始化代码

这里顺序要注意：MPU 配置在前，D-Cache 开启在后，而且 Cache 只需要在上电初始化时开启一次，不要在每次 `tiny_solve` 前反复操作。I-Cache 主要减少从 Flash 取指的等待；D-Cache 对 AXI SRAM 的数据访问有帮助。我把 TinyMPC 的大结构体放在 DTCM，访问时间相对稳定。

Cache 打开后，还要处理 DMA 缓冲的一致性。DMA 直接访问内存，如果缓冲区被 D-Cache 缓存，CPU 就可能读到旧数据。我把 DMA 缓冲统一放到 0x30000000 开始的 D2 SRAM，并通过 MPU 把这块区域设置为 non-cacheable.[1]

可以先定义一个统一的段属性，所有 DMA 缓冲都用它，不要只移动数组中的一部分：链接脚本把 `.ram_d2` 放进 `RAM_D2`，MPU 再把同一地址范围设为 non-cacheable：

MPU 区域大小和起始地址必须按芯片要求对齐，并覆盖链接脚本实际使用的 `RAM_D2` 范围。启动顺序仍然是 `MPU_Config()` 在前、`SCB_EnableDCache()` 在后。这样 CPU 开着 D-Cache 跑 TinyMPC，DMA 缓冲却始终绕开 Cache，接收完成后 CPU 读到的就是 DMA 刚写入的数据。如果项目没有划 non-cacheable 区，才需要对可缓存 DMA 缓冲做 Cache 维护：发送前 Clean，接收后 Invalidate，地址和长度都按 32 字节 Cache Line 向外对齐。两种方案选一种完整执行，不要既没有 MPU 隔离，也没有 Clean/Invalidate。

内存配置完成后，就可以初始化 TinyMPC。先将 MATLAB 导出的 `Kinf`、`Pinf`、`Quu_inv`、`AmBKt` 等数据装入 cache，再设置 `rho`、最大迭代次数、primal/dual 阈值，打开状态和输入的 box constraint，并填入 `x_min/x_max`、`u_min/u_max`。

第一次启动时，将 `slack`、`dual` 和 `x/u` 预测轨迹初始化为零，再设置 `A`、`B`、`f`、`Q`、`R` 及参考轨迹，最后把 `work`、`cache`、`settings` 挂到 solver 上。

还有一个很容易忽略的细节是 `rho` 的一致性。它既参与 MATLAB 端的离线 Riccati 缓存计算，也参与 H7 上的 ADMM 更新，所以两边必须完全对应。例如 H7 里写了 `cache.rho = 1.0f`，前提就是生成系数时 `cfg.rho` 也确实设成了 1.0。到这里，求解器就基本能接入实车控制循环了。下面这张图是我采用的任务运行流程：

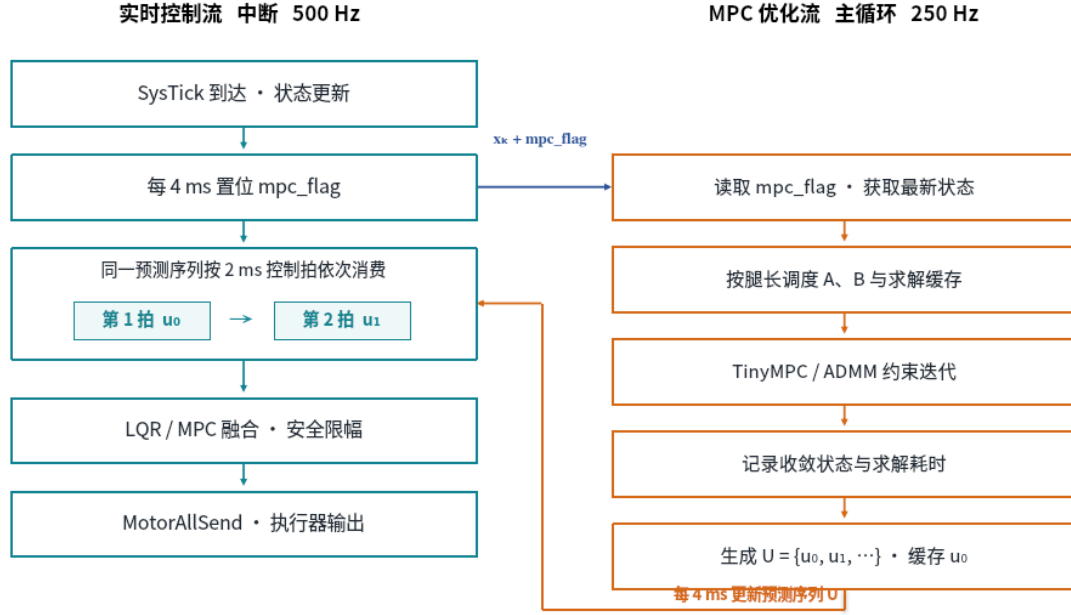


图 10 500 Hz 控制中断与 250 Hz MPC 任务流程

实际求解时，每拍更新当前状态、参考值、约束，以及随腿长变化的模型矩阵，再调用 TinyMPC。

求出的第一个控制量 u_0 还不能直接发给电机，需要先检查 `work.status`、迭代次数、`residual`、IMU 状态以及机器人是否离地。遇到数值异常或模型已经不适用的工况，就执行安全输出策略，必要时切回 LQR/PID。

六、用动量观测器识别碰撞和堵转

轮腿的侧面装甲在对抗中经常会受到撞击，有时还会被障碍物卡住。由于整车没有额外安装力传感器，我尝试利用机器人自身的动力学模型和电机反馈来判断外部扰动，最后采用了广义动量观测器 (GMO)[4]。

以五自由度 $q=[s, \phi, \text{right_leg}, \text{left_leg}, \text{pitch}]$ 描述整车，外部碰撞表现为未知广义力矩：

$$M_o(q)\ddot{q} + C_o(q, \dot{q})\dot{q} + G_o(q) = \tau + \tau_{\text{ext}},$$

$$p = M_o(q)\dot{q}, \quad \dot{p} = \tau + C_o^\top(q, \dot{q})\dot{q} - G_o(q) + \tau_{\text{ext}}.$$

这里用下标 o 区分观测器的机械动力学矩阵。观测器的 M_o 由拉格朗日模型构造，保持对称正定，并与 C_o 使用同一套坐标和惯性定义，使动量关系成立。MPC 为消去内力整理出的约化系数矩阵不能未经检查就直接拿来代替。

τ 是执行器作用映射得到的广义力，轮毂力矩由真实电流换算，关节力矩按 VMC 和方向关系组装。采用带反馈的积分观测器，可以避免直接差分动量带来的噪声：

$$\dot{\hat{p}} = \tau + C_o^\top \dot{q} - G_o + r,$$

$$r = K_o(p - \hat{p}), \quad K_o \succ 0,$$

$$\hat{p}(0) = p(0).$$

理想情况下，残差能够动态跟踪外部扰动，但实际也包含摩擦和模型误差，因此主要用于异常检测，而非精确测量接触力。

实车实现中，针对起步瞬态、稳态偏置和轮端打滑等干扰，设计了启动补偿、分轴阈值和持续时间判据，分别识别偏航堵转、腿部卡阻及纵向冲击。其中，偏航堵转通过残差阈值、轮速一致性和连续检测共同判断，减少误触发。下图为部分检测效果：

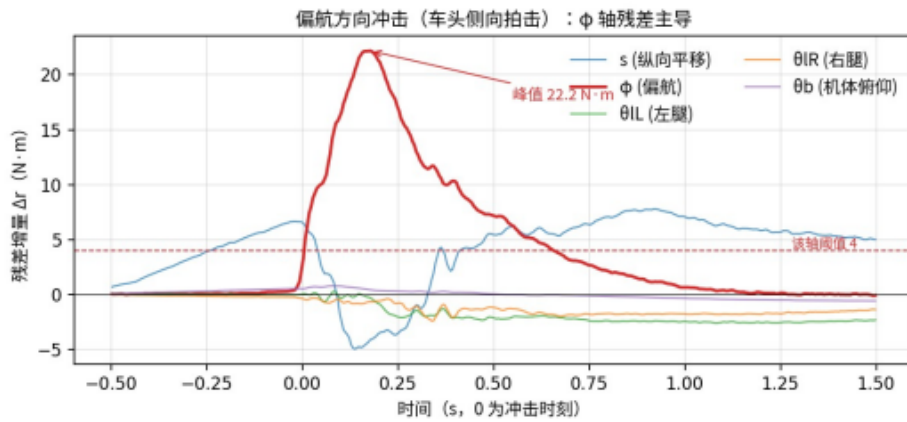


图 11 偏航方向外部冲击时的分轴残差

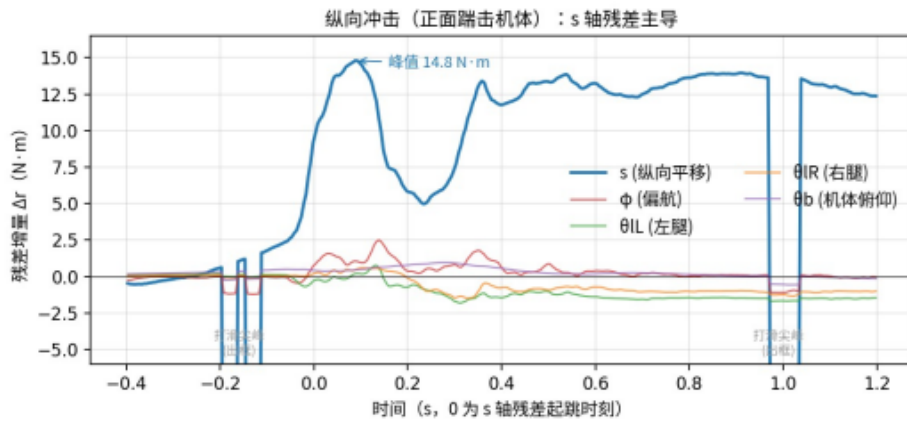


图 12 纵向冲击时的分轴残差

在此基础上，设计了两类响应策略：

偏航参考导纳：当检测到持续外力矩时，允许航向参考适当偏移，降低机器人与外力的对抗；撤力后逐渐恢复原参考。

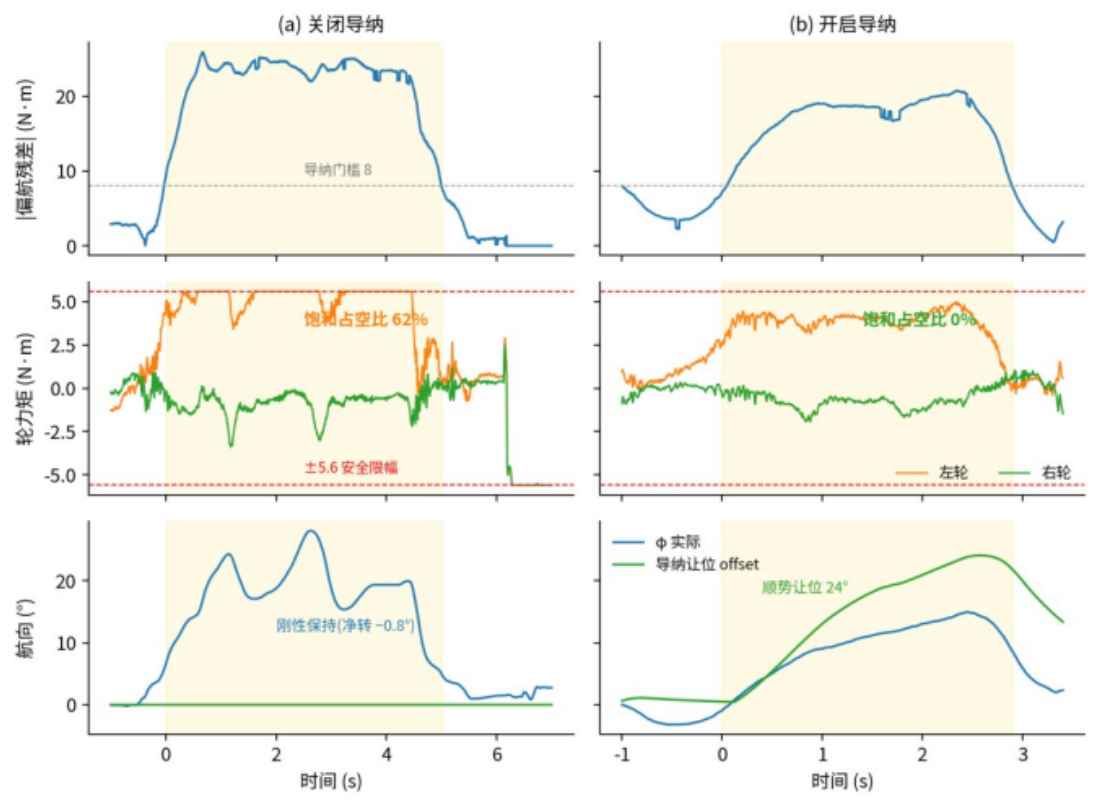


图 13 参考导纳与堵转响应的部分实车曲线

堵转脱困：当检测到持续卡阻时，触发主动旋转脱困，并设置超时退出和冷却机制。
具体见代码开源。

七、 仓库地址和交流

本项目地址： https://github.com/dexori/TinyMPC_WheelLeg.git

注：由于原代码耦合较多，不太美观这里用 ai 工具进行优化裁剪形成该项目代码。

除此之外 Control-Note 用具体实例讲解 PID、LQR、MPC 等控制方法，也介绍了离散化、嵌入式实现和实机调试，对学习控制理论、理解算法如何落到工程中很有帮助 [5]。论坛站内帖子《更适合 RM 的控制理论》也附在下面，方便大家查阅和交流。

GitHub 仓库： <https://github.com/liuskywalkerjskd/Control-Note> 。

感谢 TinyMPC 作者，也感谢论坛里分享各种轮腿方案。这篇记录的是我从力矩限幅问题出发，把 MPC 接到单片机，再加入碰撞判断的过程。模型、公式或者接法有问题欢迎指出。